# Dimensionality reduction for homological stability and global structure preservation

Alexander Kolpakov[1] and Igor Rivin[2]

[1]University of Austin, Austin TX, USA; akolpakov@uaustin.org
[2]Temple University, Philadelphia PA, USA; rivin@temple.edu

**Abstract**

We propose DiRe, a force–directed dimensionality reduction framework designed to preserve global structure and homological features while remaining practical on modern hardware. The computations in this paper use DiRe–RAPIDS, while DiRe–JAX is the earlier implementation associated with the JOSS publication. The method combines an initial embedding with a graph–based layout optimization and evaluates the resulting low–dimensional representation using local distortion, context preservation, and persistent homology measures. Across the benchmark suite considered here, DiRe provides a complementary tradeoff to UMAP and tSNE: it is designed less as a purely local visualization heuristic and more as a framework for embeddings whose large–scale geometry can be quantified through Betti curves and persistence diagrams.

## 1 Introduction

Traditional dimensionality reduction techniques such as UMAP and tSNE [MHM18; MH08] are widely used for visualizing high–dimensional data in lower–dimensional spaces, usually 2D and sometimes 3D. Other uses include dimensionality reduction to other, possibly higher and thus non–visual dimensions, for the subsequent use of classifiers such as SVMs.

However, these methods often struggle with scalability, interpretability, and preservation of global data structures. UMAP, while fast and scalable, may overemphasize local structures at the expense of global data relationships [CP23]. And tSNE, while known for producing high–quality visualizations, may be computationally expensive and sensitive to hyperparameter tuning [KB19].

DiRe addresses these challenges by combining an initial low–dimensional embedding with a force–directed layout driven by the $k$–nearest–neighbor graph of the original data. The goal is not to replace every use of UMAP or tSNE, but to provide an alternative whose optimization and diagnostics are explicitly tied to preservation of large–scale and homological structure.

The computational results in this manuscript use DiRe–RAPIDS [KR26a]. Its GPU stack includes PyTorch [Pas+19], optional PyKeOps acceleration [Cha+21], and RAPIDS/cuVS components [Dev25a; RAP26]; DiRe–JAX is the earlier implementation described in [KR25a].

The contribution of this paper is therefore twofold. First, it describes the algorithmic framework and the homological stability motivation behind DiRe. Second, it evaluates the framework with metrics that measure local distortion, classification context, and persistent homology of the original data and its embedding.

### 1.1 Relation to existing methods

tSNE and UMAP both build low–dimensional embeddings by matching neighborhood information, but they differ in objective functions, kernels, and optimization details [MH08; KB19; MHM18]. DiRe uses the $k$NN graph in a different role: it supplies the attraction graph for a force–directed layout after an explicit initial embedding. PCA and random projections provide analytical reference points because their distortion can be bounded directly in terms of variance or Johnson–Lindenstrauss distortion, while spectral methods use graph operators related to the same neighborhood graph [BN03; CL06]. DiRe is closest in spirit to graph layout methods, but the evaluation criterion emphasized here is topological: we compare persistence diagrams and Betti curves of the high–dimensional point cloud with those of the resulting embedding.

## 2 Main methods

Let $X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^D$ be the input point cloud, represented in practice as an $n \times D$ numerical array. DiRe performs the following main steps:

1. **Capturing dataset topology:** Create the kNN graph of $X$, say $\Gamma$, for a given number of neighbors $k$ (= `n_neighbors`). DiRe–RAPIDS can use PyTorch, PyKeOps, or RAPIDS/cuVS backends for this step, depending on data size and available hardware. Other libraries like FAISS [Dou+24] may also be used for nearest–neighbor search.

2. **Initial dimension reduction:** Produce $Y \subset \mathbb{R}^d$, the initial embedding of $X$, with $d \ll D$ (usually $d = 2$ or 3) specified by `dimension`. The available initial embedding methods include `random` (random projections from the Johnson–Lindenstrauss lemma), `spectral` (using the kNN graph $\Gamma$ to construct a weighted Laplacian, optionally with a similarity kernel), and `pca` (classical or kernel–based).

3. **Layout optimization:** Adjust the lower–dimensional embedding $Y$ to conform to the similarity structure of the higher–dimensional data $X$. This is done by using a force layout in which the role of "forces" is played by probability kernels; their scale is controlled by parameters such as `min_dist` and `spread`.

In DiRe–RAPIDS, users can instantiate PyTorch, memory–efficient PyTorch/PyKeOps, or cuVS backends directly; alternatively, `create_dire` selects an implementation according to available dependencies and hardware. The initial embedding and optimized layout are exposed by the software so that the effect of the layout step can be studied separately.

The initial embedding can be one of the following three choices, each of which we discuss in more detail.

## 2.1 Random Projection embedding

This embedding is based on the Johnson–Lindenstrauss lemma [JL84].

**Johnson–Lindenstrauss Lemma (Probabilistic Form)**

Given $0 < \epsilon < 1$ and an integer $n$, let $X$ be a set of $n$ points in $\mathbb{R}^d$. For a random linear map $f : \mathbb{R}^d \to \mathbb{R}^k$ where $k = O\left(\frac{\log n}{\epsilon^2}\right)$, with high probability, for all $u, v \in X$,

$$(1-\epsilon)\|u-v\|^2 \leq \|f(u)-f(v)\|^2 \leq (1+\epsilon)\|u-v\|^2.$$

The value

$$\text{dist}(f) = \frac{\|f(u)-f(v)\|}{\|u-v\|}$$

is called the *distortion* of $f$, and is expected to be close to 1.0 for a good quality embedding.

The Johnson–Lindenstrauss lemma says that a random projection from $\mathbb{R}^d$ to $\mathbb{R}^k$ has, with high probability, distortion close to 1 when $k$ is sufficiently large relative to $\log n$. This guarantee does not imply small distortion when $k$ is restricted to the visualization range ($k = 1, 2, 3$). The method is nevertheless simple and computationally inexpensive.

The image of a projection $p : \mathbb{R}^d \to \mathbb{R}^k$ may be cluttered when $k \ll d$. Indeed, the total variance of $Y = p(X) \in \mathbb{R}^{n \times k}$ may be much lower than the total variance of $X \in \mathbb{R}^{n \times d}$ unless the projection captures the dominant variance directions.

## 2.2 Principal Component Analysis embedding

The discussion above motivates projections that preserve as much variance as possible. The classical method for this purpose is Principal Component Analysis.

Assume that $X$ is centered, so that each column of $X$ has zero mean. Let its covariance matrix be $\text{Cov}(X) = \frac{1}{n-1} X^t X$. PCA is computed from the singular value decomposition

$$X = U\,\Sigma\,W^t,$$

where $\Sigma$ has $k \ll d$ dominant singular values $\Sigma_k = \langle \sigma_1, \ldots, \sigma_k \rangle$ with

$$\sum_{i=1}^{k} \sigma_i^2 = (1-\varepsilon)^2 \|X\|_F^2.$$

Let $\Sigma_k$ be the rank $k$ truncation of $\Sigma$ preserving the dominant singular values above, and let $U_k$ and $W_k$ be the respective truncations of $U$ and $W$. Then the rank $k$ approximation of $X$ is

$$\widehat{X} = U_k\,\Sigma_k\,W_k^t$$

and the PCA embedding of $X$ is

$$X_k = X\,W_k.$$

If the dominant topological features of $X$ are represented in the leading $k$ principal directions, then the persistence diagrams of $X$ can be compared with those of the projected point cloud $X_k$. This equality is exact when $X$ lies in an affine $k$–dimensional subspace of $\mathbb{R}^d$. More generally, stability of persistence diagrams gives the following bound [CSO14]:

$$d_b(D(X), D(X_k)) = d_b(D(\widehat{X}W_k), D(XW_k)) \leq \|\widehat{X}W_k - XW_k\|_F = \|\widehat{X} - X\|_F = \varepsilon\|X\|_F.$$

In this sense, PCA gives a useful reference case: the homological distortion is controlled by the low–rank approximation error when the retained coordinates capture the relevant geometry.

### 2.3 Spectral Laplacian embedding

Spectral Laplacian embeddings are commonly used for manifold learning from local graph structure [TSL00; BN03]. Diffusion maps use related graph operators to organize data by diffusion geometry [CL06]. Since the $kNN$–graph $\Gamma$ of $X$ is already available in DiRe, a Laplacian embedding can be computed with standard graph methods.

### 2.4 Force–directed layout

After the initial embedding, DiRe applies an iterative force–directed layout in order to correlate the local structure of the embedding $Y$ with the local structure of the high–dimensional dataset $X$. The forces of attraction and repulsion are given by simple distributions modeled after the $t$–distributions of UMAP and tSNE.

Namely, a simple density function

$$\varphi(x) = \frac{1}{1 + a\,\|x\|^{2b}}$$

is fit to `min_dist` $= \delta$ and `spread` $= \sigma$ parameters, so that

$$\varphi(x) \approx 1.0$$

within $\|x\| < \delta$, and then decays exponentially as

$$\varphi(x) \approx \exp(-(\|x\| - \delta)/\sigma)$$

outside of the $\delta$–ball.

Attraction forces are defined between points in $Y$ corresponding to $kNN$–neighbors in $\Gamma$, with magnitude

$$\varphi\left(\frac{1}{\|y_i - y_j\|}\right), \text{ for } y_i, y_j \in Y \text{ such that } (y_i, y_j) \in \Gamma.$$

All other pairs of points experience a repulsion force of magnitude

$$\varphi\left(\|y_i - y_j\|\right) \text{ for } y_i, y_j \in Y \text{ such that } (y_i, y_j) \notin \Gamma.$$

The layout is run for a prescribed iteration budget. Parameters such as `n_neighbors`, `min_dist`, `spread`, backend choice, random seed, and floating–point precision affect the final embedding and are part of any reproducible specification. In applications these parameters can be tuned against a chosen validation criterion, for example by Bayesian optimization packages such as Hyperopt [BYC13] or Optuna [Aki+19]. In the benchmarks below we instead use fixed, reported protocol choices so that the comparison is reproducible and not dataset–specific hyperparameter search.

# 3 Quantitative measures

We use several quantitative measures to assess the quality of DiRe embeddings, and to compare the algorithmic framework to other embedding techniques available, such as tSNE in its cuML implementation, as well as UMAP in both its original and cuML implementations.

## 3.1 Measuring the global structure

We use several types of measures for persistence homology. One type is related to *persistence diagrams*, and the other is related to the *Betti curves* derived from them. Although basically equivalent, these two types of persistence homology representation happen to be useful in different ways.

### 3.1.1 Persistence diagrams

Persistence diagrams are classical representations of the "birth–death" pairs in persistence homology. The reader may find more information in [ZC05; Zom05], while we give a brief description below.

The *Vietoris—Rips complex* at scale $t \geq 0$, denoted $\mathrm{VR}_t(X)$, is the simplicial complex defined as

$$\mathrm{VR}_t(X) = \{\sigma \subseteq X \mid \|x_i - x_j\| \leq t \text{ for all } x_i, x_j \in \sigma\},$$

which means a simplex $\sigma$ is contained in $\mathrm{VR}_t(X)$ if all its vertices are pairwise within distance $t$.

Consider a non–decreasing sequence of scales $\{t_i\}_{i \in I}$, where $I$ is an index set (often $I = \mathbb{R}_{\geq 0}$), generating a *filtration* of Rips complexes:

$$\mathrm{VR}_{t_0}(X) \subseteq \mathrm{VR}_{t_1}(X) \subseteq \mathrm{VR}_{t_2}(X) \subseteq \cdots.$$

For each $k \geq 0$ and scale $t_i$, compute the $k$-th homology group $H_k\left(\mathrm{VR}_{t_i}(X); \mathbb{F}\right)$ with coefficients in a field $\mathbb{F}$ (commonly $\mathbb{F} = \mathbb{Z}/2\mathbb{Z}$, or another finite field). The inclusion maps induce homomorphisms between homology groups:

$$f_{s,t}^k : H_k\left(\mathrm{VR}_s(X);\mathbb{F}\right) \to H_k\left(\mathrm{VR}_t(X);\mathbb{F}\right), \quad \text{for } s \le t.$$

A persistent $k$-dimensional homology class $\alpha$ is an element of $H_k\left(\mathrm{VR}_s(X);\mathbb{F}\right)$ that persists across multiple scales. The *birth time* $b_\alpha$ and *death time* $d_\alpha$ of $\alpha$ are defined as

- $b_\alpha$: the smallest $t$ where $\alpha \neq 0$ appears in $H_k\left(\mathrm{VR}_t(X);\mathbb{F}\right)$;
- $d_\alpha$: the smallest $t > b_\alpha$ where $f_{b_\alpha,t}^k(\alpha) = 0$.

For homology classes that persist indefinitely, we set their death time to infinity: $d_\alpha = \infty$.

The *persistence diagram* $D_k$ is the (multi–)set of points $(b_\alpha, d_\alpha)$ in the extended plane $\mathbb{R}^2 \cup \{\infty\}$:

$$D_k(X) = \left\{(b_\alpha, d_\alpha) \in (\mathbb{R} \times (\mathbb{R} \cup \{\infty\})) \mid \alpha \in H_k(X) \text{ with birth } b_\alpha \text{ and death } d_\alpha\right\},$$

where we use $H_k(X)$ to denote the $k$–dimensional persistence homology group defined above.

Features with longer lifespans $d_\alpha - b_\alpha$ are considered topologically significant. In particular, features with $d_\alpha = \infty$ represent persistent homological features that never disappear within the considered scale range, indicating essential topological structures of the space.

The usual metrics between two diagrams $D = D_k(X)$ and $D' = D_k(Y)$ are the bottleneck and Wasserstein distances, cf. [Per24].

### 3.1.2 Betti curves

Betti curves mark the progression of the persistence Betti numbers with the filtration level increase. This representation was first studied in detail in [GGB16]. Let $X$ be a point cloud and $\mathrm{Rips}_t(X)$ be its Rips complex of level $t \ge 0$. Then the $k$–th Betti curve is

$$\beta_k(t) = \operatorname{rank} H_k(\mathrm{VR}_t(X)).$$

Several metrics can be used to compare two Betti curves. We include distances that are insensitive to moderate reparametrisations of the filtration scale $t$.

The most common are the Dynamic Time Warp (DTW) distance [SC07; Dev23], Time Warp Edit Distance (TWED) [Mar09; Zum21], and Earth Mover's Distance (EMD) [Fla+21; POT25].

### 3.1.3 Global structure preservation

Let $X \in \mathbb{R}^{n\times d}$ be a set of $n$ vectors in $\mathbb{R}^d$ with $kNN$–graph $\Gamma$. Let $Y = f(X) \in \mathbb{R}^{n\times k}$ be its embedding. We measure standard distances between the corresponding persistence diagrams $D_k^H = D_k(X)$ and $D_k^L = D_k(Y)$ of higher–dimensional data and its lower–dimensional embedding, respectively. In particular, we use the bottleneck distance $d_b(D_k^H, D_k^L)$ and Wasserstein distance $d_W(D_k^H, D_k^L)$ for $k = 0, 1$, since the reported embeddings are 2–dimensional.

We use the normalized version of $d_b$ and $d_W$, where the distance is averaged over paired subsamples of points from $X$ and $Y$ for computational feasibility. The subsampling seed and sample size are part of the benchmark configuration.

We also employ the Betti curves $\beta^H(t)$ for the higher–dimensional data and $\beta^L(t)$ for the lower–dimensional embedding. The distances computed between $\beta^H(t)$ and $\beta^L(t)$ are DTW, TWED, and EMD.

We compute normalized versions by averaging over subsamples. For EMD, the curves are first normalized to unit mass and the distance is then rescaled. If $\tau$ is the normalizing factor, we compute $\max\{\tau, \tau^{-1}\} \cdot d_{EMD}(\beta^H, \tau \cdot \beta^L)$.

## 3.2 Measuring the local structure

We use two local measures: embedding stress and neighborhood preservation ratio.

### 3.2.1 Embedding stress

Let $X \in \mathbb{R}^{n\times d}$ be a set of $n$ vectors in $\mathbb{R}^d$ with $kNN$–graph $\Gamma$. Let $Y = f(X) \in \mathbb{R}^{n\times k}$ be its embedding. For each edge $e = (x, y)$ in $\Gamma$ we have its local embedding stress measured by

$$\lambda(e) = \left|1 - \frac{\|x - y\|_2}{\|f(x) - f(y)\|_2}\right|$$

Let $\Lambda = \Lambda(f, \Gamma) = (\lambda(e))_{e\in \mathrm{edges}(\Gamma)}$ be the sequence of local edge stresses. The total embedding stress of $\Gamma$ under $f$ is then computed as

$$\sigma(f, \Gamma) = \frac{\sqrt{\mathrm{Var}[\Lambda]}}{\mathbb{E}[\Lambda]}.$$

Note that $\sigma$ is scale–invariant in the sense that if all edge lengths of $\Gamma$ are being rescaled under $f$, then $\sigma(f, \Gamma) = 0$, as there is no *relative* change of the internal metric of $\Gamma$.

### 3.2.2 Neighborhood preservation

Given $X$ and its $kNN$ graph $\Gamma_X := \Gamma$, let us consider $Y = f(X)$ and the $kNN$ graph $\Gamma_Y$ of $Y$. The neighborhood preservation measure counts how many neighbours of a vertex $x$ in $\Gamma_X$ are mapped to neighbours of $f(x)$ in $\Gamma_Y$. That is, let $N_X(x)$ be the neighbor indices for a vertex $x$ of $\Gamma_X$. Let the vertex indices in $Y$ be inherited from $X$, that is $y_i = f(x_i)$, for $x_i$ a vertex in $\Gamma_X$. Let $N_Y(f(x))$ be the neighbor indices for the embedded point $f(x)$ in $\Gamma_Y$.

Then the neighborhood preservation sequence of $f$ is given by

$$N(X, f) = \left(\frac{\#\big(N_X(x) \cap N_Y(f(x))\big)}{\#N_X(x)}\right)_{x\in X}.$$

The neighborhood preservation index is then the pair

$$\nu(X, f) = \left(\mathbb{E}[N(X, f)], \sqrt{\mathrm{Var}[N(X, f)]}\right).$$

We intentionally compute both the mean and standard deviation in order to see not only how well neighborhoods are preserved, but also how variable the loss of local structure is.

In our numerical experiments, this quantity is interpreted together with stress and persistence metrics, since local nearest–neighbour agreement and global topological agreement need not rank the methods in the same way.

### 3.3 Measuring context loss

If $X$ is a labeled set of data points, then its *context* is the mutual position, both geometric and topological, of its labeled subsets. A useful lower–dimensional embedding should preserve enough of this context to remain informative about the high–dimensional relationships in $X$.

#### 3.3.1 Linear SVM classifier accuracy

If $X$ is labeled, we compare the accuracy of a linear Support Vector Machine trained and tested on $X$ with the corresponding accuracy for $Y$. This provides a simple measure of whether the embedding changes the linear separability of labeled subsets.

Thus, we expect the accuracy $\alpha_X$ of an SVM classifier trained and tested on $X$ to be close to that of one trained and tested on $Y$, denoted $\alpha_Y$. The SVM context loss is defined by

$$\kappa_{SVM} = \log \min \left\{ \frac{\alpha_X}{\alpha_Y}, \frac{\alpha_Y}{\alpha_X} \right\}$$

A substantial change in accuracy indicates that the embedding has changed the linear separability of labeled subsets.

#### 3.3.2 $kNN$ classifier accuracy

We also compare the accuracy of a $kNN$ classifier trained and tested on $X$ to that of one trained and tested on $Y$. This measures whether local class neighborhoods are preserved by the embedding.

Let $\alpha_X$ be the accuracy of a $kNN$ classifier trained and tested on $X$, and let $\alpha_Y$ be the accuracy of a $kNN$ classifier trained and tested on $Y$. The $kNN$ context loss is then defined as

$$\kappa_{kNN} = \log \frac{\alpha_Y}{\alpha_X}.$$

## 4 General workflow

DiRe follows a streamlined workflow that is designed for ease of use and integration into existing data analysis pipelines.

1. **Data Preprocessing:** DiRe does not prescribe a universal preprocessing pipeline. Whether the data has to be normalized, standardized, or processed via some transformation for variance reduction (e.g. log– or $\sinh^{-1}$–transform) depends on the dataset and scientific question. The benchmark suite reports preprocessing choices explicitly; for single–cell datasets, metadata columns are treated as labels or covariates rather than expression features unless stated otherwise.

2. **Data embedding:** Performed according to the steps described in the previous section (cf. Methods).

3. **Integration and Export:** DiRe–RAPIDS is designed to be integrated into larger machine learning workflows. It exposes a `fit_transform` interface, backend selection utilities, memory–efficient variants, and metric utilities for comparing embeddings.

4. **Embedding metrics:** The metrics described above make the final embedding quantitatively comparable across methods and parameter choices, and can also be used as objectives for hyperparameter optimization.

# 5 Benchmarking

A collection of benchmarks is available in the DiRe–RAPIDS repository [KR26a], some of which are given below. We concentrate on the main differences between DiRe, tSNE, and UMAP. The benchmark protocol uses DiRe–RAPIDS for DiRe, cuML implementations of tSNE and UMAP for GPU baselines [Dev25a], and the original UMAP implementation by McInnes et al. as a CPU reference implementation [Dev25b; MHM18]. Runtime comparisons therefore report backend and hardware explicitly; original `umap-learn` runtimes are not interpreted as GPU performance.

## 5.1 Benchmark protocol

All benchmark figures in this manuscript are regenerated from the same benchmark runner and archived run logs. For each dataset and method we record the dataset source, number of points, ambient dimension, preprocessing, random seed, backend, wall–clock `fit_transform` runtime, and the local, context, and persistence metrics used below. The scatter plots use a single canonical seed, while non-topological metric plots report the mean over seeded runs, with error bars equal to the sample standard deviation. Persistent homology metrics are computed on paired subsamples of the high–dimensional data and embedding; the subsample seed and fraction are part of the benchmark configuration and are implemented in the released benchmark runner. The number of repeats contributing to each plotted metric is recorded in the JSON logs. The Lambda Labs instance used for these calculations is an NVIDIA A10 instance in the `gpu_1x_a10` class; the exact driver, CUDA, and package versions are recorded at runtime.

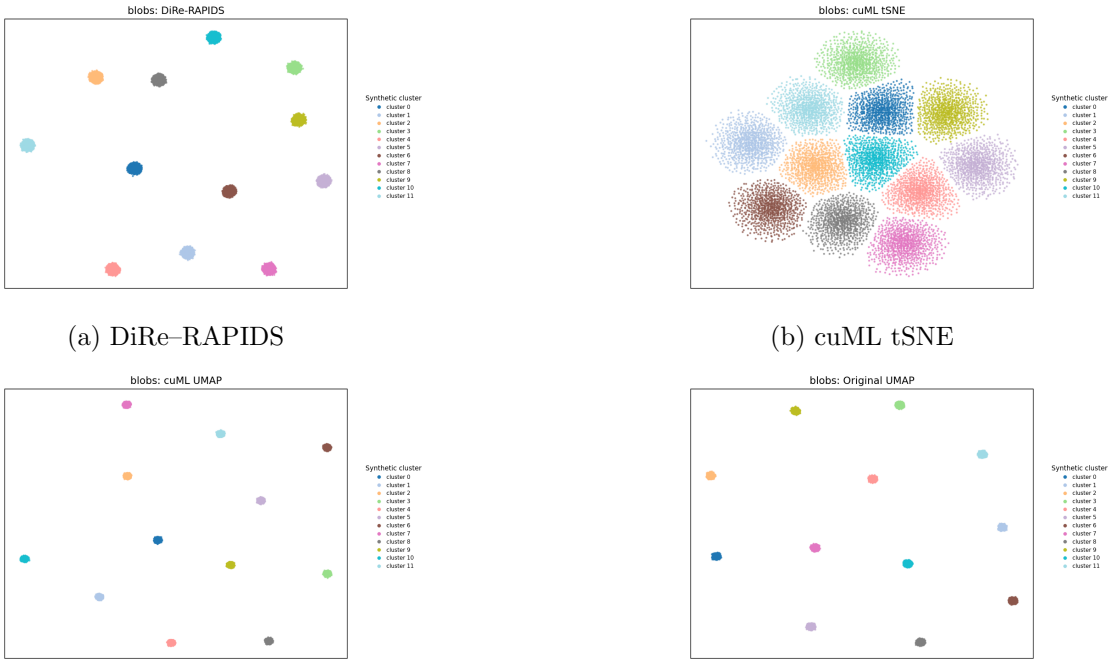
## 5.2 Dataset: Blobs

The 2D embeddings of the blobs dataset are given in Fig. 1. The dataset has 10K points, 12 Gaussian clusters, and ambient dimension 1000. It is primarily a sanity check: under appropriate parameters, all methods should recover the cluster structure.

The key metrics are given in Appendix 9. We compare the visual embedding, local metrics, and persistence metrics, since these criteria capture different aspects of the output.

The local metrics can rank the methods differently from the global metrics, which is expected because stress and neighborhood preservation measure local metric distortion rather than cluster–level topology.

The average neighborhood preservation score should not be interpreted as a complete quality measure. Even simple dimension reductions can change nearest–neighbor identities while preserving larger clusters or connected components. For example, the planar point cloud $X_k = \{x_n = (1/n, n) : n = 1, \ldots, k\}$ has $x_k$ as the point farthest from the origin, while its projection to the horizontal axis is closest to 0.




(a) DiRe–RAPIDS

(b) cuML tSNE

(c) cuML UMAP

(d) Original UMAP

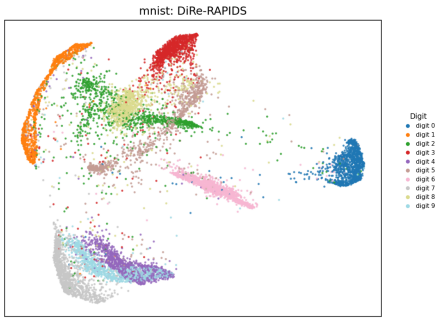
Figure 1: "Blobs" dataset embeddings

## 5.3 Dataset: MNIST Digits

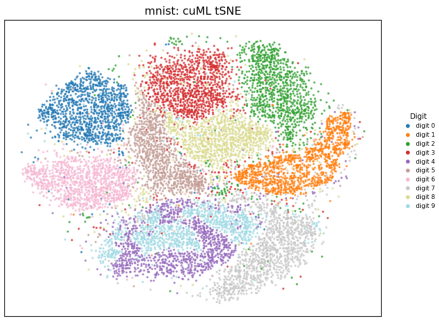
MNIST Digits provides a labeled image benchmark with well known class structure. This benchmark compares whether the methods preserve digit classes, transitions between visually similar digits, and the persistent homology summaries of the high–dimensional data.

The key metrics are given in Appendix 10. In this dataset, visual cluster separation, classifier context, and persistence distances need not induce the same ranking, so these criteria are reported separately.

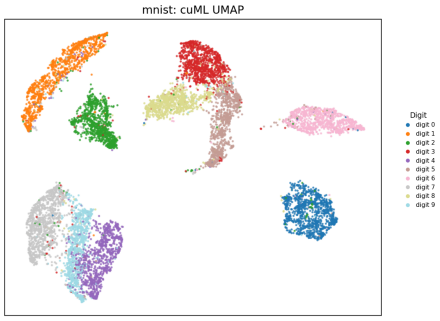
Even when all methods reasonably preserve labeled context, the SVM and kNN context scores can distinguish linear separability from local class consistency.



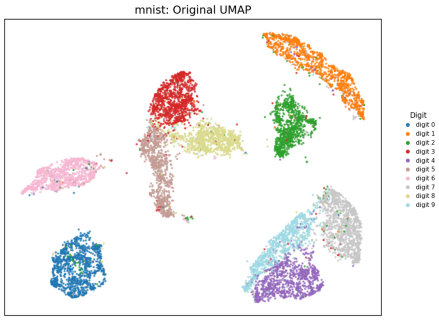

(a) DiRe–RAPIDS




(b) cuML tSNE




(c) cuML UMAP




(d) Original UMAP

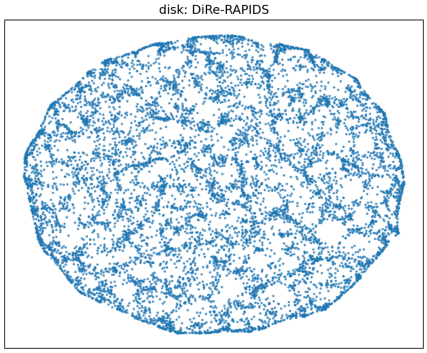
Figure 2: "MNIST Digits" dataset embeddings

### 5.4 Dataset: Disk Uniform

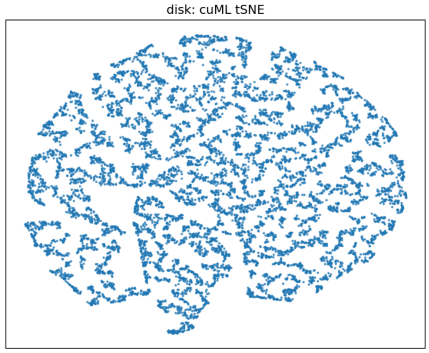
Here we embed a 2D disk with points uniformly distributed in it. Since the input already has a simple low–dimensional geometry, this benchmark tests whether a method unnecessarily distorts connectedness, density, or boundary shape.

A similar family of tests can be run on a sphere and an ellipsoid in 3D with uniformly distributed points. See the DiRe–RAPIDS repository for additional datasets [KR26a].

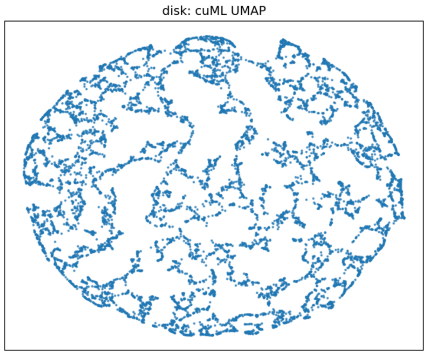
The metrics are given in Appendix 11. We treat persistence metrics and local distortion metrics as complementary: a method may preserve nearest neighbors while changing the global shape of this simple input.



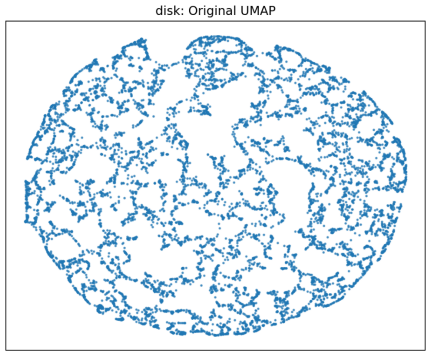

(a) DiRe–RAPIDS




(b) cuML tSNE




(c) cuML UMAP




(d) Original UMAP

Figure 3: "Disk Uniform" dataset embeddings

### 5.5 Dataset: Half-moons

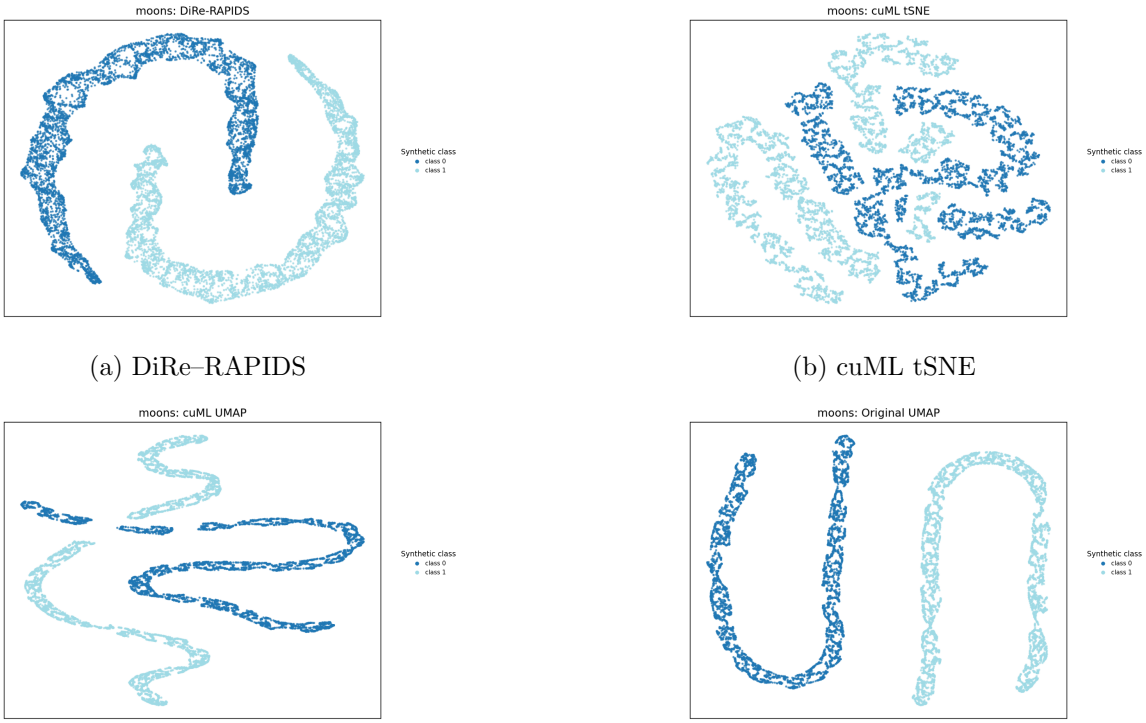
The half–moons dataset tests whether a method preserves the non–linear relation between two labeled components. Since the input classes are not linearly separable, a two–dimensional embedding that makes them linearly separable may improve a downstream classifier while changing the original context. The context preservation score is reported in Appendix 12.

We compare the embeddings against both context metrics and persistence metrics, rather than relying on visual inspection alone.

### 5.6 Dataset: Levine 13

The Levine 13 CyTOF single–cell dataset serves as a biological benchmark in which preprocessing choices matter. In this benchmark, the expression channels are transformed with an inverse hyperbolic sine transform with cofactor 5; label and individual metadata columns are excluded from the feature matrix, and unassigned cells are removed.

We report persistence and context metrics separately, since biological labels, local neighborhoods, and topological summaries need not agree perfectly.




(a) DiRe–RAPIDS

(b) cuML tSNE

(c) cuML UMAP

(d) Original UMAP

Figure 4: "Two Half–Moons" dataset embeddings

More benchmarking information is available in Appendix 13.

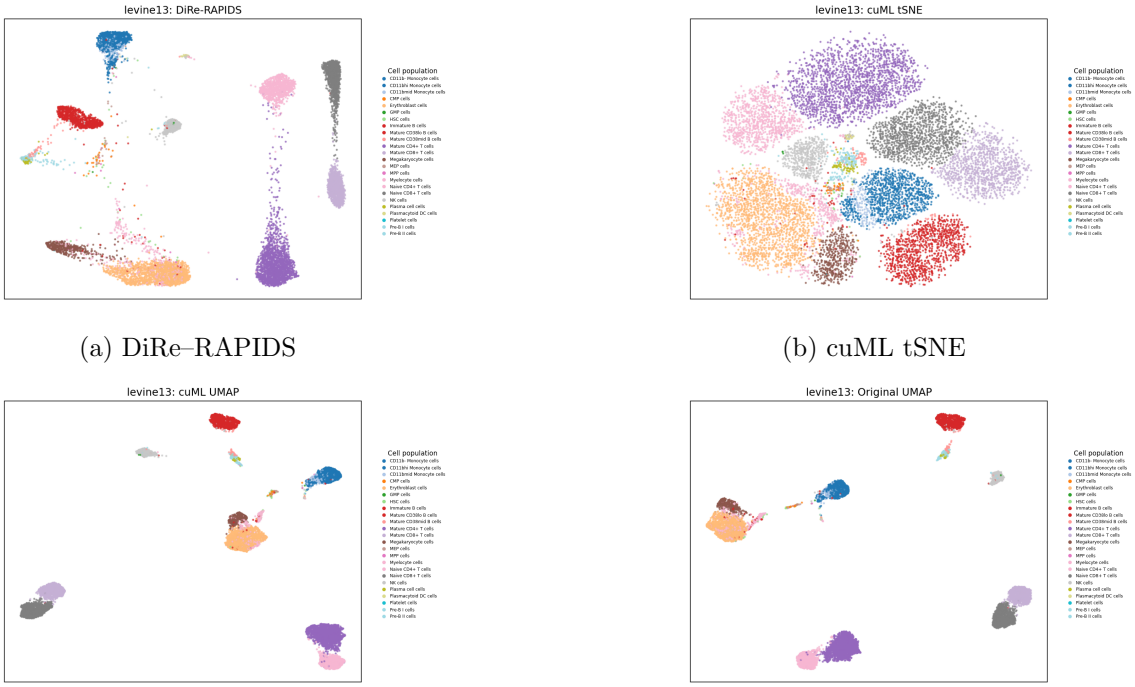
### 5.7 Dataset: Levine 32

The Levine 32 CyTOF dataset is handled with particular care because patient, batch, or other metadata columns can dominate an embedding if they are inadvertently included as numerical features. The same preprocessing protocol is used as for Levine 13: expression channels are arcsinh–transformed with cofactor 5, label and individual metadata columns are excluded from the feature matrix, and unassigned cells are removed.

More benchmarking data is available in Appendix 14.

## 6 Conclusions

A new dimensionality reduction framework, called `DiRe`, has been developed with an explicit emphasis on preserving and measuring the dataset's homological structure. The latter is quantified by persistent homology and expressed via persistence diagrams or Betti curves. The benchmarks and metrics provided give evidence that DiRe can preserve global structure in cases where purely local criteria are insufficient, while also showing that no single metric captures every scientifically relevant aspect of an embedding.




(a) DiRe–RAPIDS

(b) cuML tSNE

(c) cuML UMAP

(d) Original UMAP

Figure 5: "Levine 13" dataset embeddings

The numerical experiments in this manuscript use DiRe–RAPIDS [KR26a]; the earlier DiRe–JAX implementation remains available from GitHub [KR25b] and is described in the JOSS publication [KR25a].

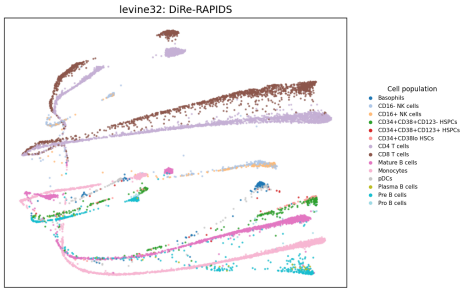
The benchmark logs, archived embedding images, plotting scripts, manuscript build scripts, and Sphinx version of this manuscript are available in a separate reproducibility repository [KR26b]. The DiRe implementation used by those scripts is DiRe–RAPIDS [KR26a]. GPU acceleration is recommended for rerunning the largest datasets and for runtime comparisons against cuML baselines.

## 7 Funding

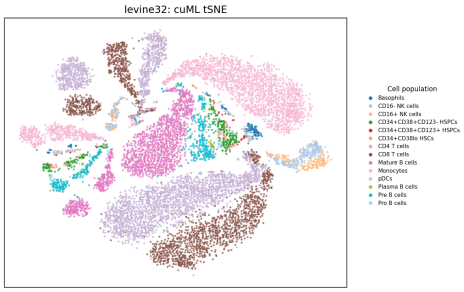
This work is supported by the Google Cloud Research Award number GCP19980904. The computational benchmarks were run in part on Lambda Labs GPU infrastructure; we thank Lambda Labs for supporting this work with access to GPU computing resources.

## 8 Author Contributions

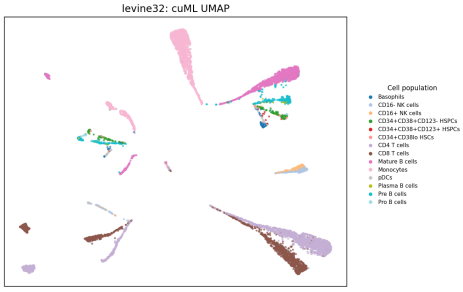
A.K. and I.R. developed the necessary theoretical background and main algorithm. A.K. produced and prepared figures. All authors reviewed the manuscript.



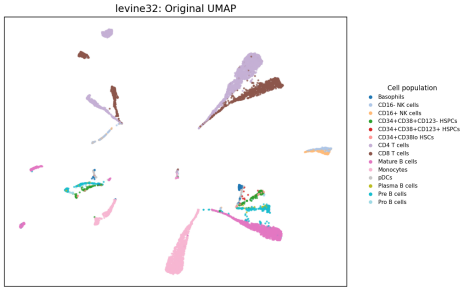

(a) DiRe–RAPIDS




(b) cuML tSNE




(c) cuML UMAP




(d) Original UMAP

Figure 6: "Levine 32" dataset embeddings

## References

[JL84] William B. Johnson and Joram Lindenstrauss. "Extensions of Lipschitz mappings into a Hilbert space". In: *Contemporary Mathematics* 26 (1984), pp. 189–206. DOI: 10.1090/conm/026. URL: https://doi.org/10.1090/conm/026.

[TSL00] Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. "A global geometric framework for nonlinear dimensionality reduction". In: *Science* 290.5500 (2000), pp. 2319–2323. DOI: 10.1126/science.290.5500.2319. URL: https://doi.org/10.1126/science.290.5500.2319.

[BN03] Mikhail Belkin and Partha Niyogi. "Laplacian Eigenmaps for dimensionality reduction and data representation". In: *Neural Computation* 15.6 (2003), pp. 1373–1396. DOI: 10.1162/089976603321780317. URL: https://doi.org/10.1162/089976603321780317.

[ZC05] Afra Zomorodian and Gunnar Carlsson. "Computing Persistent Homology". In: *Discrete & Computational Geometry* 33.2 (2005), pp. 249–274. DOI: 10.1007/s00454-004-1146-y. URL: https://doi.org/10.1007/s00454-004-1146-y.

[Zom05] Afra J. Zomorodian. *Topology for Computing*. New York, NY, USA: Cambridge University Press, 2005. DOI: 10.1017/CBO9780511546945. URL: https://doi.org/10.1017/CBO9780511546945.

[CL06] Ronald R. Coifman and Stéphane Lafon. "Diffusion maps". In: *Applied and Computational Harmonic Analysis* 21.1 (2006), pp. 5–30. DOI: 10.1016/j.acha.2006.04.006. URL: https://doi.org/10.1016/j.acha.2006.04.006.

[SC07] Stan Salvador and Philip Chan. "Toward Accurate Dynamic Time Warping in Linear Time and Space". In: *Intelligent Data Analysis* 11.5 (2007), pp. 561–580. URL: https://doi.org/10.3233/IDA-2007-11508.

[MH08] Laurens van der Maaten and Geoffrey Hinton. "Visualizing data using t-SNE". In: *Journal of Machine Learning Research* 9.86 (2008), pp. 2579–2605. URL: https://www.jmlr.org/papers/v9/vandermaaten08a.html.

[Mar09] Pierre-François Marteau. "Time Warp Edit Distance with Stiffness Adjustment for Time Series Matching". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 31.2 (2009), pp. 306–318. DOI: 10.1109/TPAMI.2008.76. URL: https://doi.org/10.1109/TPAMI.2008.76.

[BYC13] James Bergstra, Daniel Yamins, and David D. Cox. "Making a Science of Model Search: Hyperparameter Optimization in Hundreds of Dimensions for Vision Architectures". In: *Proceedings of the 30th International Conference on Machine Learning*. 2013, pp. 115–123. URL: https://proceedings.mlr.press/v28/bergstra13.html.

[CSO14] Frédéric Chazal, Vin de Silva, and Steve Oudot. "Persistence stability for geometric complexes". In: *Geometriae Dedicata* 173.1 (2014), pp. 193–214. DOI: 10.1007/s10711-013-9937-z. URL: https://doi.org/10.1007/s10711-013-9937-z.

[GGB16] Chad Giusti, Robert Ghrist, and Danielle S. Bassett. "Two's company, three (or more) is a simplex". In: *Journal of Computational Neuroscience* 41.1 (2016), pp. 1–14. DOI: 10.1007/s10827-016-0608-6. URL: https://doi.org/10.1007/s10827-016-0608-6.

[MHM18] Leland McInnes, John Healy, and James Melville. "UMAP: Uniform Manifold Approximation and Projection". In: *Journal of Open Source Software* 3.29 (2018), p. 861. DOI: 10.21105/joss.00861. URL: https://doi.org/10.21105/joss.00861.

[Aki+19] Takuya Akiba et al. "Optuna: A Next-generation Hyperparameter Optimization Framework". In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019, pp. 2623–2631. DOI: 10.1145/3292500.3330701. URL: https://doi.org/10.1145/3292500.3330701.

[KB19] Dmitry Kobak and Philipp Berens. "The art of using t-SNE for single-cell transcriptomics". In: *Nature Communications* 10.1 (2019), p. 5416. DOI: 10.1038/s41467-019-13056-x. URL: https://doi.org/10.1038/s41467-019-13056-x.

[Pas+19] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.

[Cha+21] Benjamin Charlier et al. "Kernel operations on the GPU, with autodiff, without memory overflows". In: *Journal of Machine Learning Research* 22.74 (2021), pp. 1–6. URL: http://jmlr.org/papers/v22/20-275.html.

[Fla+21] Rémi Flamary et al. "POT: Python Optimal Transport". In: *Journal of Machine Learning Research* 22.78 (2021), pp. 1–8. URL: http://jmlr.org/papers/v22/20-451.html.

[Zum21] Marcus Voßand Jonathan Zumer. *PyTWED: Python wrapper for Time Warped Edit Distance*. https://github.com/jzumer/pytwed. 2021.

[CP23] Tara Chari and Lior Pachter. "The specious art of single-cell genomics". In: *PLoS Computational Biology* 19.8 (2023), e1011288. DOI: 10.1371/journal.pcbi.1011288. URL: https://doi.org/10.1371/journal.pcbi.1011288.

[Dev23] FastDTW Developers. *FastDTW: A Python Implementation of Fast Dynamic Time Warping*. https://github.com/slaypni/fastdtw. 2023.

[Dou+24] Matthijs Douze et al. *The FAISS library*. 2024. arXiv: 2401.08281 [cs.LG].

[Per24] Persim Developers. *Persim: Distances and representations of persistence diagrams*. https://github.com/scikit-tda/persim. 2024.

[Dev25a] Rapids.ai cuML Developers. *cuML - GPU Machine Learning Algorithms*. https://github.com/rapidsai/cuml/. 2025.

[Dev25b] UMAP Developers. *Uniform Manifold Approximation and Projection (UMAP)*. https://github.com/lmcinnes/umap/. 2025.

[KR25a] Alexander Kolpakov and Igor Rivin. "DiRe- JAX: A JAX based Dimensionality Reduction Algorithm for Large-scale Data". In: *Journal of Open Source Software* 10.110 (2025), p. 8264. URL: https://doi.org/10.21105/joss.08264.

[KR25b] Alexander Kolpakov and Igor Rivin. *DiRe: Dimensionality Reducer*. https://github.com/sashakolpakov/dire-jax. 2025.

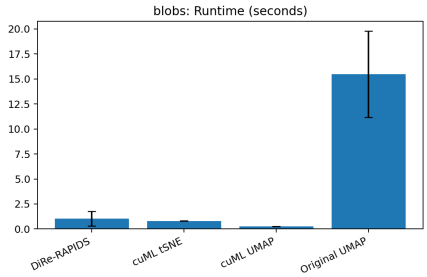
[POT25] POT Developers. *POT: Python Optimal Transport*. https://github.com/PythonOT/POT. 2025.

[KR26a] Alexander Kolpakov and Igor Rivin. *DiRe-RAPIDS: PyTorch and RAPIDS accelerated dimensionality reduction*. https://github.com/sashakolpakov/dire-rapids. 2026.

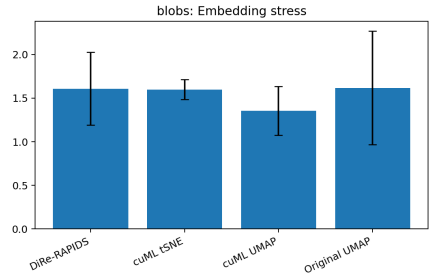
[KR26b] Alexander Kolpakov and Igor Rivin. *Homological stability reproducibility package*. https://github.com/sashakolpakov/homological-stability-repro. 2026.

[RAP26] RAPIDS cuVS Developers. *cuVS: Vector search and clustering on the GPU*. https://github.com/rapidsai/cuvs. 2026.

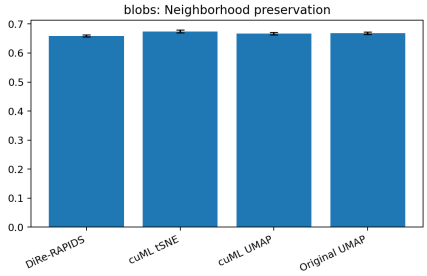
# 9 Blobs: Dataset Metrics



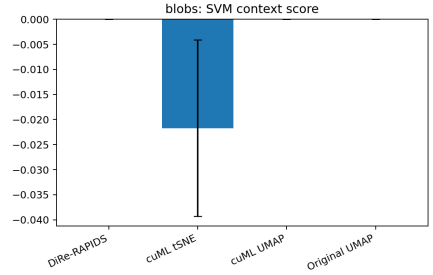

(a) Runtime (seconds)



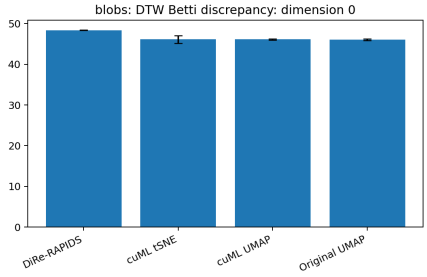

(b) Embedding stress



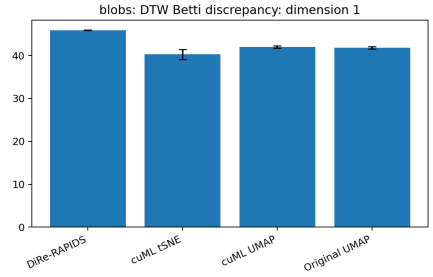

(c) Neighborhood preservation




(d) SVM context score



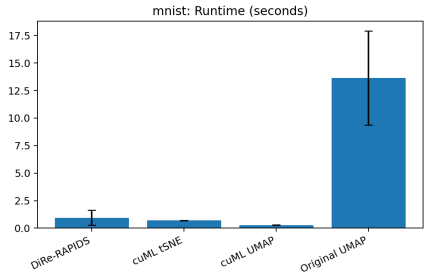

(e) DTW Betti discrepancy: dimension 0



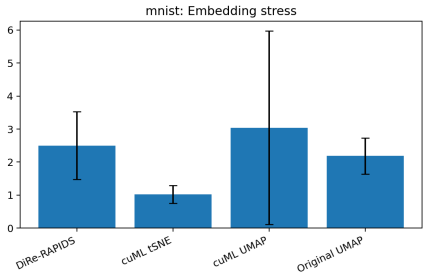

(f) DTW Betti discrepancy: dimension 1

Figure 7: "Blobs" dataset metrics comparison. Error bars show sample standard deviations.

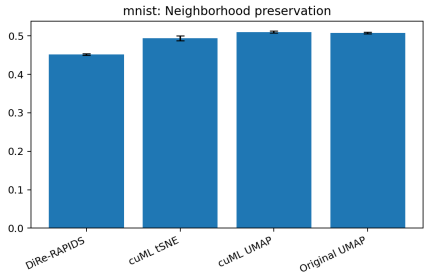
# 10 MNIST Digits: Dataset Metrics



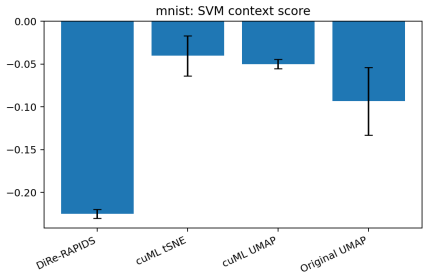

(a) Runtime (seconds)



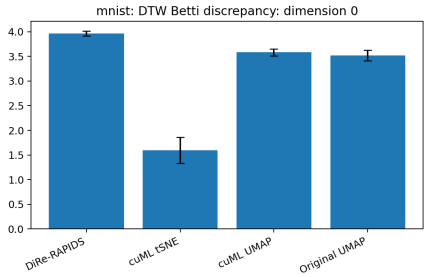

(b) Embedding stress



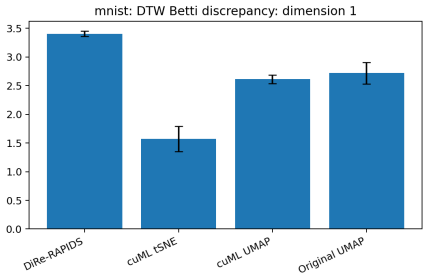

(c) Neighborhood preservation




(d) SVM context score



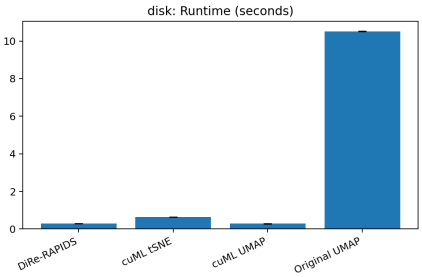

(e) DTW Betti discrepancy: dimension 0



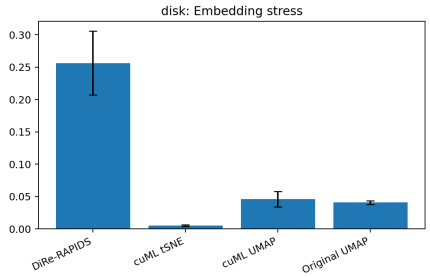

(f) DTW Betti discrepancy: dimension 1

Figure 8: "MNIST Digits" dataset metrics comparison. Error bars show sample standard deviations.

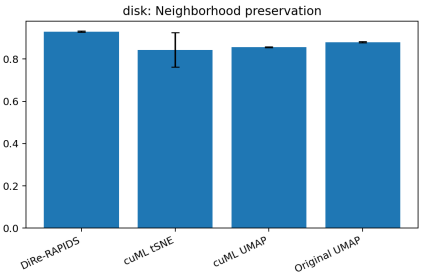
# 11 Disk Uniform: Dataset Metrics



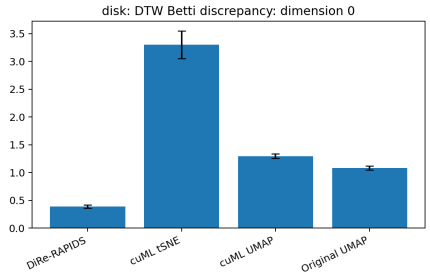

(a) Runtime (seconds)



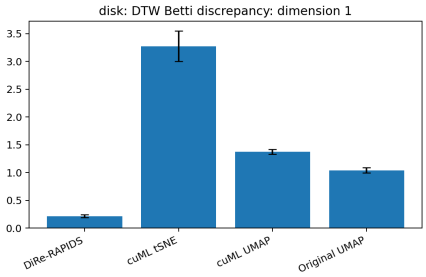

(b) Embedding stress




(c) Neighborhood preservation



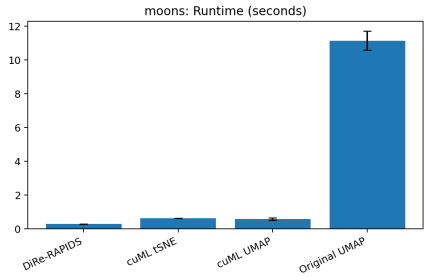

(d) DTW Betti discrepancy: dimension 0



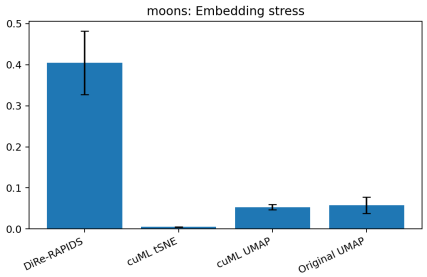

(e) DTW Betti discrepancy: dimension 1

Figure 9: "Disk Uniform" dataset metrics comparison. Error bars show sample standard deviations.

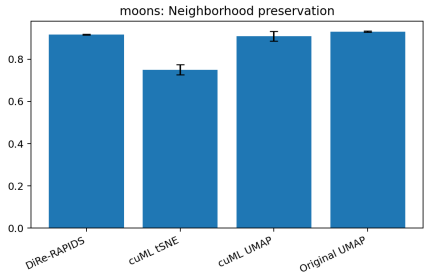
# 12 Half–moons: Dataset Metrics



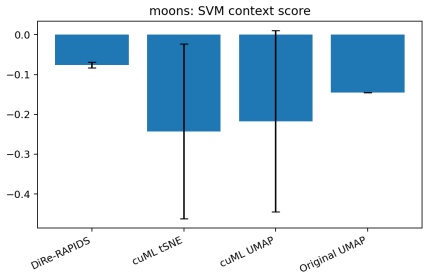

(a) Runtime (seconds)



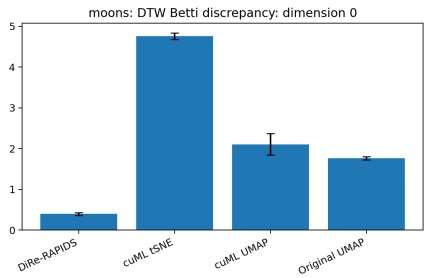

(b) Embedding stress



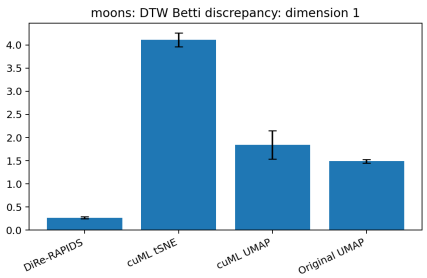

(c) Neighborhood preservation




(d) SVM context score



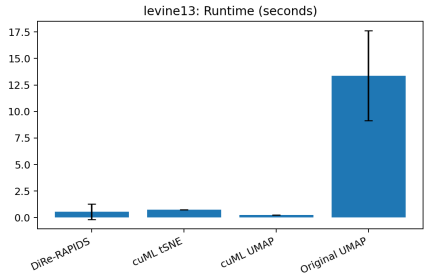

(e) DTW Betti discrepancy: dimension 0



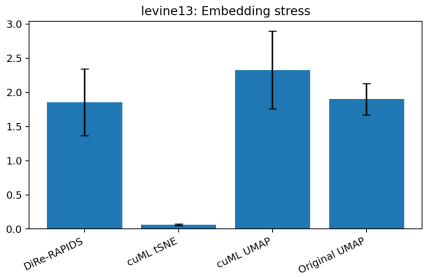

(f) DTW Betti discrepancy: dimension 1

Figure 10: "Half–moons" dataset metrics comparison. Error bars show sample standard deviations.

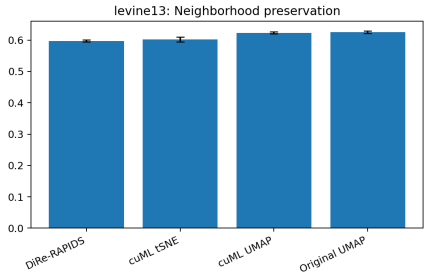
# 13 Levine 13: Dataset Metrics



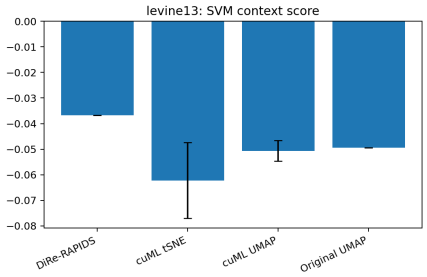

(a) Runtime (seconds)



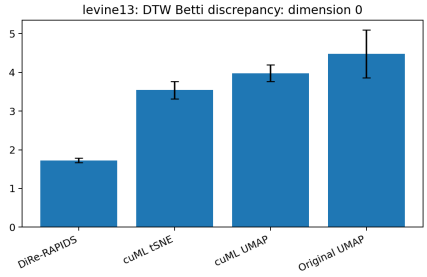

(b) Embedding stress



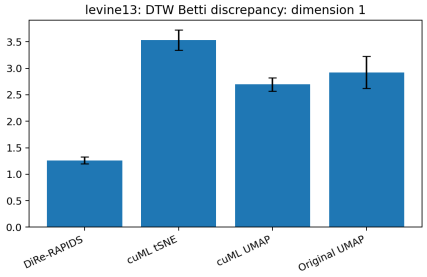

(c) Neighborhood preservation




(d) SVM context score



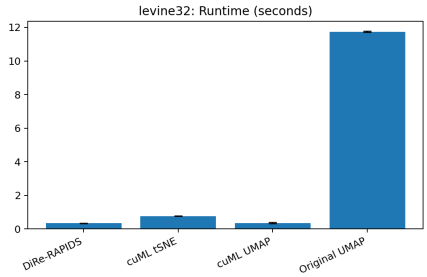

(e) DTW Betti discrepancy: dimension 0



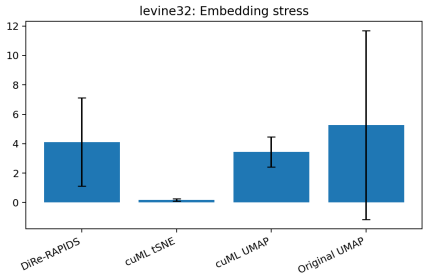

(f) DTW Betti discrepancy: dimension 1

Figure 11: "Levine 13" dataset metrics comparison. Error bars show sample standard deviations.

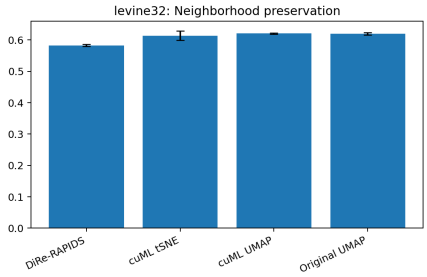
# 14 Levine 32: Dataset Metrics



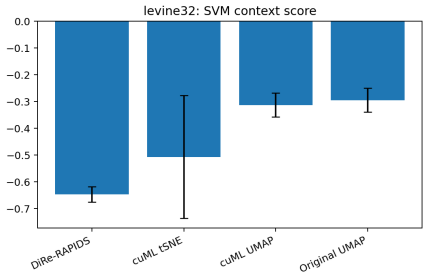

(a) Runtime (seconds)



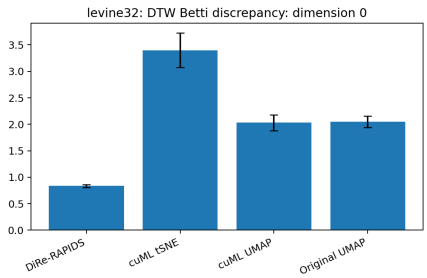

(b) Embedding stress



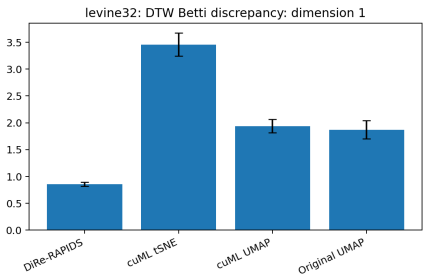

(c) Neighborhood preservation




(d) SVM context score




(e) DTW Betti discrepancy: dimension 0




(f) DTW Betti discrepancy: dimension 1

Figure 12: "Levine 32" dataset metrics comparison. Error bars show sample standard deviations.